

Relatório de Aprendizagem – Integração Web

Neste projeto, foi desenvolvida uma aplicação web para cadastro de clientes, utilizando **HTML**, **CSS**, **Python com Flask**, **MySQL** e a biblioteca **mysql.connector**. O objetivo principal foi criar um sistema simples em que o usuário preenche um formulário na página web e os dados são enviados para um banco de dados.

O projeto foi organizado em três partes principais: o arquivo **app.py**, responsável pela lógica da aplicação; o arquivo **index.html**, responsável pela estrutura da página; o arquivo **style.css**, responsável pela aparência visual; e também os comandos SQL utilizados para criar o banco de dados e a tabela.

Estrutura do projeto

O projeto foi organizado da seguinte forma:

formpy/

|

├── app.py

|

├── templates/

| └── index.html

|

└── static/

└── style.css

O Flask utiliza essa organização porque, por padrão, os arquivos HTML ficam dentro da pasta `templates`, enquanto os arquivos CSS ficam dentro da pasta `static`.

Utilização do HTML

O arquivo `index.html` foi utilizado para criar a estrutura da página de cadastro de clientes.

Nele foi criado um formulário com os seguintes campos:

```
<input type="text" id="cpf" name="cpf" maxlength="11" required />
```

```
<input type="text" id="primeiro_nome" name="primeiro_nome" required />
```

```
<input type="text" id="sobrenome" name="sobrenome" required />
```

```
<input type="number" id="idade" name="idade" required />
```

Esses campos permitem que o usuário informe o **CPF**, o **primeiro nome**, o **sobrenome** e a **idade**.

O formulário foi definido com a seguinte linha:

```
<form action="/cadastrar" method="post">
```

Essa linha indica que, quando o botão **Cadastrar** for pressionado, os dados serão enviados para a rota `/cadastrar`, usando o método `POST`.

O método `POST` é usado quando queremos enviar dados do formulário para o servidor, nesse caso, para o código Python no arquivo `app.py`.

Outro ponto importante é o atributo `name` dos campos. Por exemplo:

```
name="cpf"
```

Esse atributo é usado pelo Flask para identificar e recuperar o valor digitado pelo usuário.

Utilização do CSS

O arquivo `style.css` foi utilizado para melhorar a aparência da página.

No CSS, foram definidos estilos para o corpo da página, título, formulário, campos de entrada e botão.

No `body`, foi aplicado um fundo com gradiente:

```
background: linear-gradient(to right, #eaf6fb, #d9eef7);
```

Esse comando cria um fundo com transição de cores, deixando a página mais agradável visualmente.

O formulário recebeu largura, margem, espaçamento interno, cor de fundo, borda arredondada e sombra:

```
form {  
  
  width: 360px;  
  
  margin: 40px auto;  
  
  padding: 30px;  
  
  background-color: #f7fcff;  
  
  border: 1px solid #cfe7f3;  
  
  border-radius: 12px;  
  
  box-shadow: 0 4px 12px rgba(80, 130, 160, 0.15);  
  
}
```

Também foram criados efeitos visuais nos campos e no botão. O `input:focus` altera a aparência do campo quando ele é selecionado pelo usuário. Já o `button:hover` muda a cor do botão quando o mouse passa sobre ele.

Utilização do Flask

O Flask foi utilizado como framework principal do projeto em Python. Ele permite criar aplicações web de forma simples, trabalhando com rotas, páginas HTML e dados enviados por formulários.

No início do arquivo `app.py`, foi usada a seguinte linha:

```
from flask import Flask, render_template, request, redirect
```

Essa linha importa recursos importantes do Flask:

- `Flask`: utilizado para criar a aplicação web;
- `render_template`: utilizado para carregar arquivos HTML da pasta `templates`;
- `request`: utilizado para receber os dados enviados pelo formulário;
- `redirect`: foi importado, mas neste código ainda não foi utilizado.

A aplicação Flask foi criada com a linha:

```
app = Flask(__name__)
```

Essa linha cria o objeto principal da aplicação. A variável `app` passa a representar o sistema web criado com Flask.

O `__name__` indica ao Flask qual é o arquivo principal da aplicação. Isso ajuda o Flask a localizar corretamente os arquivos HTML e os arquivos estáticos, como o CSS.

Criação do dicionário de configuração

No código Python, foi criado um dicionário chamado `db_config`:

```
db_config = {  
  
    'host': 'localhost',
```

```
'user': 'root',  
  
'password': 'escola',  
  
'database': 'cadastro'  
  
}
```

Esse dicionário armazena as informações necessárias para conectar o Python ao banco de dados MySQL.

As informações são organizadas em pares de chave e valor:

- `'host': 'localhost'` indica que o banco está no próprio computador;
- `'user': 'root'` indica o usuário do MySQL;
- `'password': 'escola'` indica a senha usada para acessar o banco;
- `'database': 'cadastro'` indica o banco de dados que será utilizado.

Depois, esse dicionário é usado nesta linha:

```
conexao = mysql.connector.connect(**db_config)
```

O `**db_config` envia automaticamente as informações do dicionário para a função de conexão com o MySQL.

Utilização do `mysql.connector`

O código utiliza a biblioteca `mysql.connector` para permitir a conexão entre o Python e o banco de dados MySQL.

A importação foi feita com:

```
import mysql.connector
```

Essa biblioteca permite que o Python execute comandos no banco de dados, como inserir, consultar, atualizar ou excluir registros.

No projeto, ela foi usada para conectar ao banco `cadastro` e inserir os dados do cliente na tabela `cliente`.

A conexão com o banco foi feita com:

```
conexao = mysql.connector.connect(**db_config)

cursor = conexao.cursor()
```

A variável `conexao` representa a ligação entre o Python e o banco de dados. O `cursor` é usado para executar comandos SQL.

Uso do `@app.route()`

O `@app.route()` foi utilizado para criar as rotas da aplicação. As rotas indicam o que deve acontecer quando o usuário acessa determinado endereço no navegador.

A primeira rota criada foi:

```
@app.route('/')
```

```
def index():
```

```
    return render_template('index.html')
```

Essa rota representa a página inicial do sistema. Quando o usuário acessa a aplicação, o Flask executa a função `index()` e exibe o arquivo `index.html`.

A segunda rota foi:

```
@app.route('/cadastrar', methods=['POST'])
```

```
def cadastrar():
```

Essa rota é executada quando o usuário envia o formulário. O parâmetro `methods=[ 'POST' ]` indica que essa rota aceita dados enviados pelo formulário.

Dentro dessa função, os dados digitados pelo usuário são coletados:

```
cpf = request.form.get('cpf')
```

```
nome = request.form.get('primeiro_nome')
```

```
sobrenome = request.form.get('sobrenome')
```

```
idade = request.form.get('idade')
```

Cada linha pega uma informação enviada pelo formulário HTML. Por exemplo, `request.form.get('cpf')` recupera o valor digitado no campo que possui `name="cpf"`.

Utilização do MySQL

O MySQL foi utilizado como banco de dados do projeto. Ele foi responsável por armazenar os dados dos clientes cadastrados pelo formulário.

Primeiro, foi criado o banco de dados:

```
create database cadastro;
```

Depois, o banco foi selecionado:

```
use cadastro;
```

Em seguida, foi criada a tabela `cliente`:

```
create table cliente(
```

```
CPF varchar(11) not null primary key,
```

```
PRIMEIRO_NOME varchar(20) not null,
```

```
SOBRENOME varchar(50) not null,
```

```
IDADE int not null
```

```
);
```

Essa tabela possui quatro campos:

- **CPF**: armazena o CPF do cliente, com até 11 caracteres. Ele também é a chave primária da tabela;
- **PRIMEIRO_NOME**: armazena o primeiro nome do cliente;
- **SOBRENOME**: armazena o sobrenome do cliente;
- **IDADE**: armazena a idade do cliente.

O campo **CPF** foi definido como **primary key**, ou seja, ele identifica cada cliente de forma única. Isso significa que não podem existir dois clientes com o mesmo CPF na tabela.

O comando:

```
select * from cliente;
```

foi utilizado para consultar todos os registros cadastrados na tabela **cliente**.

Inserção dos dados no banco de dados

Depois que o formulário é enviado, o Python recebe os dados e prepara um comando SQL para inserir essas informações no banco.

O comando utilizado foi:

```
comando = "INSERT INTO cliente (CPF, PRIMEIRO_NOME, SOBRENOME, IDADE)  
VALUES (%s, %s, %s, %s)"
```

```
valores = (cpf, nome, sobrenome, idade)
```

O comando **INSERT INTO** serve para inserir um novo registro na tabela **cliente**.

Os **%s** são marcadores de posição. Eles serão substituídos pelos valores armazenados na variável **valores**.

Depois, o comando é executado com:

```
cursor.execute(comando, valores)
```

```
conexao.commit()
```


O `cursor.execute()` executa o comando SQL. O `conexao.commit()` confirma a alteração no banco de dados, salvando definitivamente o cadastro.

Após a inserção, o cursor e a conexão são fechados:

```
cursor.close()
```

```
conexao.close()
```

Isso é importante para encerrar corretamente a comunicação com o banco de dados.

Tratamento de erros

O código utiliza uma estrutura `try` e `except` para tratar possíveis erros durante a conexão ou inserção dos dados.

```
try:
```

```
...
```

```
except mysql.connector.Error as err:
```

```
    return f"Erro ao cadastrar: {err}"
```

O bloco `try` tenta executar o cadastro normalmente. Caso ocorra algum erro relacionado ao MySQL, o bloco `except` captura esse erro e mostra uma mensagem na tela.

Se tudo ocorrer corretamente, o sistema retorna a mensagem:

```
return "Cliente cadastrado com sucesso!"
```

Essa mensagem informa ao usuário que os dados foram inseridos no banco com sucesso.

Execução da aplicação

No final do arquivo `app.py`, foi utilizada a seguinte estrutura:

```
if __name__ == '__main__':  
  
    app.run(debug=True)
```

Essa parte faz com que a aplicação seja executada quando o arquivo `app.py` for iniciado diretamente.

O `app.run(debug=True)` inicia o servidor Flask em modo de desenvolvimento. O `debug=True` ajuda durante a criação do projeto, pois mostra erros com mais detalhes e atualiza a aplicação automaticamente quando o código é alterado.

Funcionamento geral do projeto

O funcionamento da aplicação acontece da seguinte forma:

1. O usuário acessa a página inicial da aplicação.
2. O Flask executa a rota `/` e carrega o arquivo `index.html`.
3. O usuário preenche o formulário com CPF, primeiro nome, sobrenome e idade.
4. Ao clicar em **Cadastrar**, os dados são enviados para a rota `/cadastrar`.
5. O Flask recebe os dados usando `request.form.get()`.
6. O Python se conecta ao banco de dados MySQL usando `mysql.connector`.
7. Os dados são inseridos na tabela `cliente`.
8. Se o cadastro for realizado com sucesso, aparece a mensagem “**Cliente cadastrado com sucesso!**”.
9. Caso ocorra algum erro, o sistema mostra uma mensagem informando o problema.

Conclusão

Com este projeto, foi possível compreender como funciona a integração entre uma página web, uma aplicação em Python e um banco de dados MySQL.

O HTML foi responsável por criar o formulário de cadastro. O CSS foi utilizado para estilizar a página e melhorar a aparência visual. O Flask foi responsável por controlar as rotas, exibir a página HTML e receber os dados enviados pelo formulário. O MySQL foi usado para criar o banco de dados e armazenar as informações dos clientes. Já o `mysql.connector` permitiu a comunicação entre o Python e o MySQL.

Dessa forma, o projeto demonstrou, na prática, como uma aplicação web pode receber dados do usuário, processá-los com Python e armazená-los em um banco de dados relacional.